Pandas is a Python package providing fast, flexible, and expressive data structures designed to make working with structured (tabular, multidimensional, potentially heterogeneous) and time series data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, real world data analysis in Python.

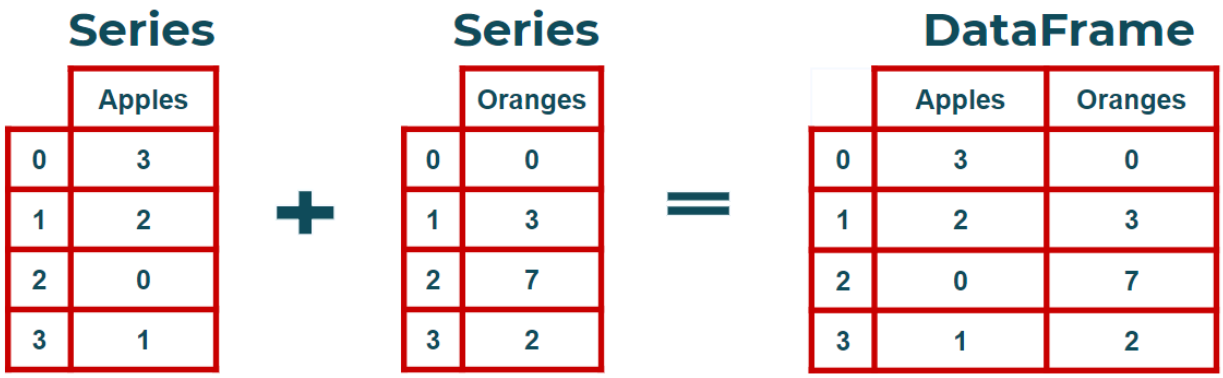
```
#Import package
import pandas as pd
import numpy as np
```

```
# Version of pandas
pd.__version__
```

```
'2.2.2'
```

There are two central concepts which you should know about:

- **Series** : A `series` is essentially a column
- **Data Frame** : A `DataFrame` is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns. Pandas DataFrame consists of three principal components, the data, rows, and columns.



Let us go ahead and create our first Pandas Series

```
# Our first series
number_series = pd.Series([2, 3, 5, 6, 8])
print(number_series)
```

```
0    2
1    3
2    5
3    6
4    8
dtype: int64
```

```
# Same series but with different indices
number_series = pd.Series([2, 3, 5, 6, 8],index=['a', 'b', 'd','e','f'],name='Number')
print(number_series)
```

```
a    2
b    3
d    5
e    6
f    8
Name: Number, dtype: int64
```

```
print(number_series[0])
print(number_series['a'])
```

```
2
2
/tmp/ipython-input-1067065090.py:1: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future
  print(number_series[0])
```

Let us go ahead and create our first Pandas DataFrame

Using a 2-D NumPy array

```
# Creating our first DataFrame
some_2D_array = np.random.randint(4, 6,(3,4))     #4 → lower bound (inclusive), 6 → upper bound (exclusive)
first_dataframe = pd.DataFrame(some_2D_array , columns=['Akshay','Devashish','Divy','Monika'])
first_dataframe
```

	Akshay	Devashish	Divy	Monika
0	4	4	5	4
1	4	4	5	5
2	5	5	4	5

```
# Lets create the same dataframe but different indices
another_dataframe = pd.DataFrame(some_2D_array, columns=['A','B','C','D'],index = ['First','Second','Third'
another_dataframe
```

	A	B	C	D
First	4	4	5	4
Second	4	4	5	5
Third	5	5	4	5

Using a conventional list of lists

```
# Initialize list of lists
list_of_lists = [['Amar', 10], ['Akbar', 15], ['Anthony', 14]]

# Create the pandas DataFrame
df = pd.DataFrame(list_of_lists, columns = ['Name', 'Age'])

# Print dataframe
df
```

	Name	Age
0	Amar	10
1	Akbar	15
2	Anthony	14

Using a dictionary

```
# Store the data in a dictionary
employee_dict = {'Employee Name':['Rajeev', 'Sumit', 'Aviral'], 'Income':[200000, 140000, 90000]}

# Create DataFrame
employee_df = pd.DataFrame(employee_dict)

# Print the dataframe
employee_df
```

	Employee Name	Income
0	Rajeev	200000
1	Sumit	140000
2	Aviral	90000

```
# Store the data in a dictionary
series_dict = {'First Series' : pd.Series([10, 20, 30, 40]),
               'Second Series' : pd.Series([10, 20, 30, 40])}

# Create the Dataframe using the dictionary
df = pd.DataFrame(series_dict)

# Print the dataframe
df
```

	First Series	Second Series
0	10	10
1	20	20
2	30	30
3	40	40

Using a list of dictionaries

```
# Initialise the list of dictionaries
list_of_dicts = [{'a': 1, 'b': 2, 'c':3}, {'a':10, 'b': 20, 'c': 30}]

# Create the DataFrame
df = pd.DataFrame(list_of_dicts)

# Print the dataframe
df
```

	a	b	c
0	1	2	3
1	10	20	30

```
# What happens if the number of keys are different in the two dictionaries
list_of_dicts = [{'a': 2, 'c':3}, {'a': 10, 'b': 20, 'c': 30}]
df = pd.DataFrame(list_of_dicts, index =['first', 'second'])
df
```

	a	c	b
first	2	3	NaN
second	10	30	20.0

There are other ways too!

```
# List of names
actor_names = ['Ryan Reynolds', 'Benedict Cumberbatch', 'Robert Downey Jr.', 'Chris Evans']

# List of their ages
actor_ages = [48,62,54,np.nan ]

# Get the list of tuples by zipping the two lists together
list_of_tuples = list(zip(actor_names, actor_ages))

# Converting the list of tuples into pandas Dataframe
actor_df = pd.DataFrame(list_of_tuples, columns = ['Name', 'Age'])

# Print the dataframe
print(list_of_tuples)
print(actor_df)
```

```
[('Ryan Reynolds', 48), ('Benedict Cumberbatch', 62), ('Robert Downey Jr.', 54), ('Chris Evans', nan)]
                  Name   Age
0        Ryan Reynolds  48.0
1  Benedict Cumberbatch  62.0
2     Robert Downey Jr.  54.0
3          Chris Evans   NaN
```

```
import numpy as np
import pandas as pd
```

```
# List of names
actor_names = ['Ryan Reynolds', 'Benedict Cumberbatch', 'Robert Downey Jr.', 'Chris Evans']

# List of their ages
actor_ages = [48,62,54]

# Get the list of tuples by zipping the two lists together
list_of_tuples = list(zip(actor_names, actor_ages))

# Converting the list of tuples into pandas Dataframe
actor_df = pd.DataFrame(list_of_tuples, columns = ['Name', 'Age'])

# Print the dataframe
actor_df
```

	Name	Age
0	Ryan Reynolds	48
1	Benedict Cumberbatch	62
2	Robert Downey Jr.	54

```
# Create lists for students and subjects
# Sample data
data = {
    "Student Name": [
        "Aarav", "Diya", "Kabir", "Meera", "Ishaan",
        "Ananya", "Rohan", "Sanya", "Arjun", "Priya"
    ],
    "Math": [85, 90, 78, 92, 88, 76, 95, 89, 84, 91],
    "Science": [80, 85, 82, 95, 87, 79, 93, 90, 86, 88],
    "English": [78, 92, 81, 89, 84, 77, 90, 85, 83, 91]
}

# Creating DataFrame
df = pd.DataFrame(data)
df
```

	Student Name	Math	Science	English
0	Aarav	85	80	78
1	Diya	90	85	92
2	Kabir	78	82	81
3	Meera	92	95	89
4	Ishaan	88	87	84
5	Ananya	76	79	77
6	Rohan	95	93	90
7	Sanya	89	90	85
8	Arjun	84	86	83
9	Priya	91	88	91

```
# Create lists for students and subjects
students = [f"Student_{i}" for i in range(1, 11)]
subjects = ["Math", "Science", "English"]

# Random marks between 70 and 100
marks = np.random.randint(70, 101, size=(10, 3))
```

```
# Create DataFrame
df2 = pd.DataFrame(marks, columns=subjects)
df2.insert(0, "Student Name", students)

# Print neatly
print("Random Student Marks DataFrame:\n")
print(df2)
```

```
Random Student Marks DataFrame:

  Student Name  Math  Science  English
0    Student_1    73       89       80
1    Student_2    77       72       79
2    Student_3    98       81       94
3    Student_4    95       87       96
4    Student_5    97       77       99
5    Student_6    97       86       73
6    Student_7    78       93       77
7    Student_8    89       83       73
8    Student_9    79       99       88
9   Student_10    97       93       77
```

```
import pandas as pd
import numpy as np

# 20 Student Names
students = [
    "Aarav", "Diya", "Kabir", "Meera", "Ishaan", "Ananya", "Rohan", "Sanya", "Arjun", "Priya",
    "Vihaan", "Nisha", "Aditya", "Kavya", "Manav", "Ritika", "Karan", "Simran", "Dev", "Pooja"
]

# Random marks between 70 and 100 for 3 subjects
marks = np.random.randint(70, 101, size=(20, 3))

# Create DataFrame
df20 = pd.DataFrame(marks, columns=["Math", "Science", "English"])
df20.insert(0, "Student Name", students)

# Save as CSV
df20.to_csv("students_20.csv", index=False)

print("CSV file 'students_20.csv' created successfully!")
```

```
CSV file 'students_20.csv' created successfully!
```

So we have seen a variety of ways in which we can create simple dataframes. Now let us explore few methods and attributes associated with the pandas dataframes in Python

We can print the indices of our dataframe using the `index` attribute

```
list(actor_df.index)
```

```
[0, 1, 2, 3]
```

```
import pandas as pd
import numpy as np
```

Let us create a dataframe with 100 rows and 5 columns

```
# Create an array of shape (100,5)
our_array = np.random.randn(100, 5)
df = pd.DataFrame(our_array, columns=['A','B','C','D','E'])
df
```

	A	B	C	D	E
0	-1.002648	1.254921	0.401186	-0.173128	0.609799
1	1.067975	1.984589	0.867378	0.482801	1.853462
2	-2.130913	-1.141573	2.594229	0.365022	-0.122008
3	-0.212262	-0.989645	-0.105038	-0.395440	-2.028866
4	-0.071954	0.414093	-0.022704	2.318046	0.720144
...	...	...	...	...	...
95	-0.125925	0.675372	-1.367703	1.956927	-0.639844
96	0.773247	-1.201248	-0.715000	-0.188179	0.221165
97	-1.170729	-0.757123	0.664607	-0.059384	-0.820009
98	-0.622138	0.536978	-1.198238	-1.204371	1.683465
99	-0.082710	-0.526076	-0.393555	0.328210	0.281828

100 rows × 5 columns

```
df.shape
```

```
(100, 5)
```

• `head()`

Prints the first n rows of the dataframe. By default it prints the first 5 rows

```
# We can use the head() method
df.head()
```

	A	B	C	D	E
0	-1.002648	1.254921	0.401186	-0.173128	0.609799
1	1.067975	1.984589	0.867378	0.482801	1.853462
2	-2.130913	-1.141573	2.594229	0.365022	-0.122008
3	-0.212262	-0.989645	-0.105038	-0.395440	-2.028866
4	-0.071954	0.414093	-0.022704	2.318046	0.720144

```
# In fact we can also print the first 10 rows
df.head(10)
```

	A	B	C	D	E
0	0.998540	-0.681302	0.824713	0.789033	1.951583
1	0.238781	0.627103	2.028656	2.154612	-1.373475
2	-0.408715	-0.081217	-0.066749	0.791193	-1.253309
3	-0.238866	-1.191920	0.151293	-0.018926	0.545097
4	-1.235711	-0.343166	0.052834	-1.081276	-1.226532
5	0.856505	-1.002563	-0.766141	-0.546868	0.132345
6	2.134486	0.994872	1.401285	-0.072825	1.294910
7	-0.422087	-1.407157	0.817301	0.215873	0.357692
8	-1.155592	0.628976	-0.108234	2.461707	-1.303632
9	-0.637359	-1.580765	-0.866442	-0.730814	0.009424

• `tail()`

Prints the last n rows of the dataframe. By default it prints the last 5 rows

```
df.tail()
```

	A	B	C	D	E
95	-1.028034	-1.368653	1.274180	0.114932	0.317147
96	-0.747171	2.621862	1.286898	-1.927143	-1.687380
97	1.742313	1.320486	0.169990	-0.335689	-1.548857
98	-1.035575	-1.042364	-0.719476	0.876073	1.092623
99	-0.529269	0.483023	1.569298	-0.931579	0.211891

```
df.tail(10)
```

	A	B	C	D	E
90	-0.680857	-0.759012	0.882982	0.520608	0.782002
91	-0.251996	0.269566	0.015296	-0.264500	1.201463
92	-0.733517	0.096039	0.384295	-0.545323	0.130120
93	1.603104	-0.088885	0.408019	-1.479782	-0.233210
94	-0.476428	0.003197	-0.203323	1.498993	0.060587
95	-1.028034	-1.368653	1.274180	0.114932	0.317147
96	-0.747171	2.621862	1.286898	-1.927143	-1.687380
97	1.742313	1.320486	0.169990	-0.335689	-1.548857
98	-1.035575	-1.042364	-0.719476	0.876073	1.092623
99	-0.529269	0.483023	1.569298	-0.931579	0.211891

- **`columns`**

Prints the list of columns in a dataframe

```
list(df.columns)
```

```
['A', 'B', 'C', 'D', 'E']
```

- **`info()`**

Prints the basic information about our dataframe

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   A       100 non-null    float64
 1   B       100 non-null    float64
 2   C       100 non-null    float64
 3   D       100 non-null    float64
 4   E       100 non-null    float64
dtypes: float64(5)
memory usage: 4.0 KB
```

```
df.describe()
```

	A	B	C	D	E
count	100.000000	100.000000	100.000000	100.000000	100.000000
mean	-0.130421	-0.100311	-0.039374	0.031499	-0.055550
std	0.997937	1.203138	1.039417	0.994481	1.015356
min	-2.381955	-2.876768	-3.102725	-2.205630	-2.779085
25%	-0.835998	-0.923389	-0.628702	-0.618226	-0.771046
50%	-0.136219	-0.145358	0.043995	0.055958	-0.030015
75%	0.616096	0.665002	0.606128	0.620531	0.584459
max	1.869165	2.721451	2.783343	2.318046	2.332184

```
actor_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   Name    4 non-null      object 
 1   Age     3 non-null      float64
dtypes: float64(1), object(1)
memory usage: 196.0+ bytes
```

• `rename()`

Rename the columns of the dataframe

```
actor_df
```

	Name	Age
0	Ryan Reynolds	48.0
1	Benedict Cumberbatch	62.0
2	Robert Downey Jr.	54.0
3	Chris Evans	NaN

```
actor_df.rename(columns = {'Name':'Actor Name','Age' :'Actor Age'}, inplace=True)
```

```
actor_df
```

	Actor Name	Actor Age
0	Ryan Reynolds	48.0
1	Benedict Cumberbatch	62.0
2	Robert Downey Jr.	54.0
3	Chris Evans	NaN

```
employee_df
```

	Employee Name	Income
0	Rajeev	200000
1	Sumit	140000
2	Aviral	90000

```
employee_df.rename(columns = {'Employee Name':'Name'}, inplace=True)
```

```
employee_df
```

	Name	Income
0	Rajeev	200000
1	Sumit	140000
2	Aviral	90000

```
employee_df = employee_df.rename({'Employee Name':'Name'},axis=1)
```

```
employee_df
```

	Name	Income
0	Rajeev	200000
1	Sumit	140000
2	Aviral	90000

Up till now, we have been creating our own simple dataframes and exploring them. But of course life is not that simple. Most of the data you would deal with in your jobs would be a lot bigger and messier. Let us start with first loading these data in our Python environment.

Loading a CSV (Comma-Separated Values) file

```
from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen — Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
sales_df=pd.read_csv("company_sales_data.csv")
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
working_dir_path = '/content/drive/MyDrive/Srishti/10 days/day 2/Python DS/company_sales_data.csv'
sales_df=pd.read_csv(working_dir_path)
```

```
sales_df = pd.read_csv('/content/drive/MyDrive/Srishti/10 days/day 2/Python DS/company_sales_data.csv')
```

```
sales_df
```

	month_number	facecream	facewash	toothpaste	bathingsoap	shampoo	moisturizer	total_units	total_profit
0	1	2500	1500	5200	9200	1200	1500	21100	211000
1	2	2630	1200	5100	6100	2100	1200	18330	183300
2	3	2140	1340	4550	9550	3550	1340	22470	224700
3	4	3400	1130	5870	8870	1870	1130	22270	222700
4	5	3600	1740	4560	7760	1560	1740	20960	209600
5	6	2760	1555	4890	7490	1890	1555	20140	201400
6	7	2980	1120	4780	8980	1780	1120	29550	295500
7	8	3700	1400	5860	9960	2860	1400	36140	361400
8	9	3540	1780	6100	8100	2100	1780	23400	234000
9	10	1990	1890	8300	10300	2300	1890	26670	266700
10	11	2340	2100	7300	13300	2400	2100	41280	412800
11	12	2900	1760	7400	14400	1800	1760	30020	300200

```
sales_df.shape
```

```
(12, 9)
```

```
sales_df.head()
```

	month_number	facecream	facewash	toothpaste	bathingsoap	shampoo	moisturizer	total_units	total_profit
0	1	2500	1500	5200	9200	1200	1500	21100	211000
1	2	2630	1200	5100	6100	2100	1200	18330	183300
2	3	2140	1340	4550	9550	3550	1340	22470	224700
3	4	3400	1130	5870	8870	1870	1130	22270	222700
4	5	3600	1740	4560	7760	1560	1740	20960	209600

```
sales_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   month_number  12 non-null     int64
 1   facecream     12 non-null     int64
 2   facewash      12 non-null     int64
 3   toothpaste    12 non-null     int64
 4   bathingsoap   12 non-null     int64
 5   shampoo       12 non-null     int64
 6   moisturizer   12 non-null     int64
 7   total_units   12 non-null     int64
 8   total_profit  12 non-null     int64
dtypes: int64(9)
memory usage: 996.0 bytes
```

```
sales_df.describe()
```

	month_number	facecream	facewash	toothpaste	bathingsoap	shampoo	moisturizer	total_units	total_profit
count	12.000000	12.000000	12.000000	12.000000	12.000000	12.000000	12.000000	12.00000	12.000000
mean	6.500000	2873.333333	1542.916667	5825.833333	9500.833333	2117.500000	1542.916667	26027.50000	260275.000000
std	3.605551	584.595172	316.733745	1242.032486	2348.095779	617.724931	316.733745	7014.36594	70143.659404
min	1.000000	1990.000000	1120.000000	4550.000000	6100.000000	1200.000000	1120.000000	18330.00000	183300.000000
25%	3.750000	2460.000000	1305.000000	4862.500000	8015.000000	1795.000000	1305.000000	21065.00000	210650.000000
50%	6.500000	2830.000000	1527.500000	5530.000000	9090.000000	1995.000000	1527.500000	22935.00000	229350.000000
75%	9.250000	3435.000000	1765.000000	6400.000000	10045.000000	2325.000000	1765.000000	29667.50000	296675.000000
max	12.000000	3700.000000	2100.000000	8300.000000	14400.000000	3550.000000	2100.000000	41280.00000	412800.000000